Week 7 Lab Handout

API Test — Automated Tests and Partner Test Exchange

Time: 110 minutes

Bring: your laptop, your working API from Weeks 5–6, `openapi.yaml`, and Node's `jest` + `supertest` packages installed

---

Before you start: your own tests only check what you thought to check

> Every check you've done so far — Weeks 5 and 6's "verify, don't eyeball" — was written and run by the same person: you. That's not a flaw, but it has a ceiling. You can only test the ways you imagine your API being used, and a bug that lives in a blind spot you don't know you have will pass every test you write, every time.

That's why today has two halves, not one. First you write your own automated tests — turning Weeks 5–6's manual checks into code that runs itself. Then you exchange with your ring partner, and let *their* tests — written from your contract, not your assumptions — run against your real API. Only one of those two halves can catch a bug you didn't know to look for.

Part A — Quick Setup (10 min)

1. Confirm your Week 5–6 API still runs.
2. Install the testing tools:

```
npm install --save-dev jest supertest
```

3. Create a `__tests__` folder at your project root, and one test file per resource (e.g. `__tests__/tasks.test.js`).
4. List every endpoint you need to cover today: every GET, POST, PUT/PATCH, and DELETE from your contract.

Part B — Write Automated Tests for Your Own Endpoints (35 min)

For every endpoint, write tests following the pattern from today's lecture: **arrange** (set up what the test needs), **act** (call the real endpoint), **assert** (check the response against your contract).

```
const request = require('supertest');
const app = require('../app');

describe('GET /tasks/:id', () => {
it('returns 404 for a task that does not exist', async () => {
const res = await request(app).get('/tasks/9999');
expect(res.status).toBe(404);
expect(res.body).toHaveProperty('message');
});
});
```

Cover all of these for every endpoint, not just the happy path

- The happy path — a normal, valid request
- Validation rejection — missing fields, wrong types (write endpoints)
- The not-found case — a real ID that doesn't exist
- At least one edge case — an empty result set, a boundary value

Write strong assertions, not weak ones

A test that only checks the status code can pass on badly broken code. Check the actual shape of the response, not just that *a* response came back:

```
// Weak — passes even if fields are named wrong
expect(res.status).toBe(200);

// Strong — actually catches contract drift
expect(res.body[0]).toHaveProperty('dueDate');
expect(typeof res.body[0].dueDate).toBe('string');
```

Run your suite before moving on:

```
npx jest
```

Every test should pass against your current, correct implementation. If something fails, that's real information — fix it now, before Part C.

Part C — Exchange With Your Ring Partner (25 min)

1. Send your `contract` (`openapi.yaml`) — not your test file — to your ring partner. They should write tests from the contract alone, the same way a real consumer would, with no access to your assumptions about how it's "supposed" to work.
2. Receive your partner's contract in return, and write your own tests against it — the same discipline, applied to their API.
3. Swap running access (a shared URL, or walk to their machine) so you can actually **run** the tests you wrote against their real, running server — and they run theirs against yours.

This is not a thought exercise. A test that was never executed against a real response has told you nothing.

Part D — Fix What Partner Testing Reveals (25 min)

Go through every failure your partner's tests surfaced. For each one, decide which of these it actually is:

- **A real bug.** Your implementation doesn't match your own contract. Fix it.
- **A contract ambiguity.** Your partner's test made a reasonable assumption your contract didn't rule out, and your implementation made a different one. Clarify the contract so there's only one reasonable reading.
- **A genuine misunderstanding.** Your partner's test expects something your contract never promised. Explain it to them directly — this isn't a bug on either side, just worth a quick conversation.

Do the same for what your tests found in *their* API — this step runs both directions.

Part E — Finalize (15 min)

Create `PARTNER_TEST_RESULTS.md` at your repo root. Document:

1. What your partner's tests found when run against your API (including "nothing — all passed," which is a real and valid outcome, not a gap).
2. What you fixed, and what you flagged as a contract ambiguity or misunderstanding instead.

Commit and push:

```
git add . __tests__ PARTNER_TEST_RESULTS.md
git commit -m "Week 7: automated tests + partner test exchange"
git push
```

(Still no branch-naming convention to follow — that starts once you're working out of the template repository your instructor hands you.)

This week's deliverable checklist

- Every endpoint has automated tests covering happy path, validation, and not-found cases
- Assertions check actual response shape, not just status codes
- Contracts exchanged with your ring partner (not test files)
- Partner's tests actually executed against your real, running API
- Every failure classified as a real bug, a contract ambiguity, or a misunderstanding
- `PARTNER_TEST_RESULTS.md` created and accurate
- All code committed and pushed

Rubric (45 pts)

Criterion	Points
Automated tests cover happy path, validation failures, and not-found cases for every endpoint	15
Tests use meaningful assertions that would actually fail on broken code	10
Partner test exchange completed, with real cross-testing evidence	10
Issues found through partner testing addressed or clearly documented	10

Common problems and quick fixes

Problem	Fix
All your tests pass, but you're not sure they'd ever fail	Deliberately break your own code for a moment (rename a field, remove a check) and re-run your tests — if nothing fails, your assertions are too weak

Problem	Fix
Partner's tests won't run against your server	Confirm your server is actually running and reachable at the URL you shared — this is a setup problem, not a code problem, almost every time
Not sure if a partner-test failure is a bug or a misunderstanding	Re-read your own contract as if you'd never seen your code — if a reasonable person could read it the way your partner's test did, it's an ambiguity, not just their mistake
Tests pass locally but you haven't re-run them since Week 5	That's mistake #5 from today's lecture — a suite that isn't re-run isn't testing anything happening now, only what was true once
Writing tests feels like duplicating work you already did manually	It is the same checks — the difference is these run themselves, every time, without you remembering to redo them by hand

Next week: can someone who has never spoken to you understand and use what you've built? Documentation begins.